Comparative Performance Analysis of CPU and GPU Ray Tracing Implementations using CUDA

Dumitru Zotea

University of Milan – MSc Computer Science

zotea.dumitru@studenti.unimi.it

October 2025

Abstract

This report presents a comparative analysis of CPU and GPU performance for a custom ray tracing system implemented in C++ and CUDA. The objective is to quantify the effect of massive GPU parallelism on render-time scalability across image resolutions. Both implementations share scene description, camera model, shading pipeline, and random-seed handling to ensure a fair comparison. The CPU renderer computes ray–object intersections sequentially; the GPU assigns one pixel per CUDA thread in a 2D launch grid and writes gamma-encoded RGB to a device framebuffer. Benchmarks are run on an NVIDIA RTX 4090 (CC 8.9) and an AMD Ryzen 9 9900X using a Cornell Box scene at 640×480, 1280×720, 1920×1080, and 3840×2160. Each configuration is repeated across five runs and averaged; outputs are validated via CRC checksums to ensure determinism. Results indicate a consistent multi-order-of-magnitude speedup, peaking at approximately $\sim$**7,500**× for the GPU path at 4K, highlighting the benefits of SIMT parallelism and coalesced memory access.

Keywords: CUDA, GPU Computing, Ray Tracing, Parallelism, Performance Comparison

1 Introduction

Ray tracing is a physically based rendering technique that simulates the behavior of light as it interacts with objects in a virtual scene. By tracing rays from the camera through each pixel and computing their intersections, reflections, and refractions, the algorithm produces images that closely approximate real-world lighting and shadows. However, the technique is computationally expensive: each pixel may require evaluating multiple rays, intersections, and material responses, leading to billions of arithmetic operations for a single frame.

This high arithmetic intensity, coupled with the independence of pixel computations, makes ray tracing an ideal candidate for **data-parallel architectures** such as modern GPUs. Unlike CPUs, which rely on a limited number of complex cores optimized for sequential workloads, GPUs expose thousands of lightweight threads capable of executing similar instructions concurrently. Frameworks like NVIDIA CUDA allow developers to map per-pixel or per-ray computations directly onto these threads, significantly reducing rendering time for the same visual result.

The goal of this project is to implement and analyze two functionally equivalent ray tracing pipelines—one running on the CPU and one on the GPU using CUDA—to quantify the benefits of massive parallelism for this class of workloads. Both versions share identical shading logic, scene setup, and output routines to ensure that observed differences stem purely from hardware execution characteristics.

Specifically, this study examines how performance scales with image resolution, evaluates kernel efficiency and memory behavior on the GPU, and contrasts these findings with the CPU's sequential execution model. The results highlight how architectural factors such as thread-level parallelism, memory coalescing, and kernel launch configuration contribute to the observed acceleration, providing insight into how parallelization transforms the runtime profile of ray tracing from a compute-bound task into a highly scalable, real-time capable process.

2 Methodology

2.1 Implementation Overview

The system is modular C++/CUDA with mirrored CPU and GPU paths to ensure parity:

- **CPU renderer** (host-only TU): loops over pixels, generates primary rays, computes closest intersections against active geometry, and shades using the shared surface shader.
- **GPU renderer**: launches a 2D grid of blocks; each thread shades one pixel by (i) generating a primary ray, (ii) tracing the closest hit using device geometry buffers, (iii) shading (Lambert + optional reflections), and (iv) writing RGBA to a device framebuffer.

The shading logic (*Lambertian diffuse* with optional soft shadows and fixed recursion depth for reflections) is shared across backends so that timing differences reflect hardware execution rather than algorithmic divergence.

2.2 Scene, Camera, and Lighting

Scenes are built via a bitmask-driven *world builder* that composes:

- Cornell Box walls (quads),
- Optional primitives (spheres, boxes constructed from reusable quad generators),
- Material presets (diffuse, mirror, refractive) with unified parameter tables.

A default pinhole camera provides FOV and ray generation. A single direct light drives Lambert shading; soft shadows (when enabled) use stratified visibility sampling. The background color is applied on miss.

2.3 CUDA Launch and Memory Model

The GPU uses a 2D launch configuration:

$$\texttt{block} = (16, 16), \quad \texttt{grid} = \left(\left\lceil \frac{W}{16} \right\rceil, \left\lceil \frac{H}{16} \right\rceil \right),$$

balancing occupancy and coalesced writes to the output surface. Geometry buffers are uploaded once per run. Threads follow a uniform control flow (fixed recursion cap) to reduce divergence. Timing uses CUDA events for GPU sections and `std::chrono` for CPU.

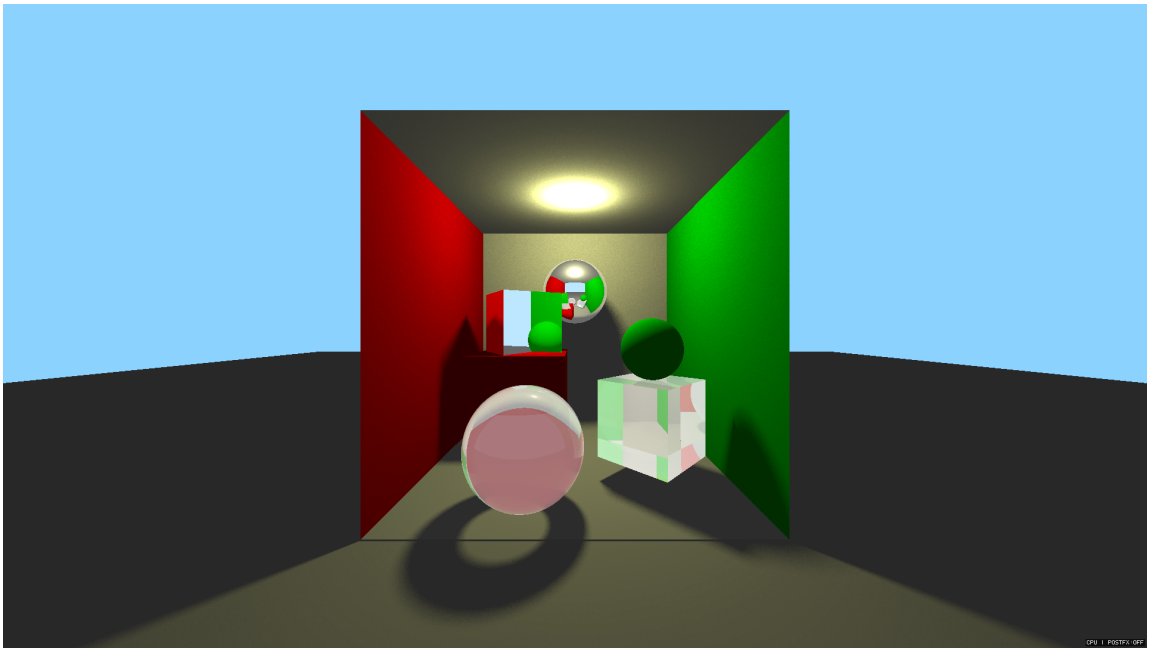

Figure 1: Cornell Box with spheres and cubes rendered at 1920×1080 using the GPU path (tone-mapped sRGB). This is the configuration used in the timing tables.

2.4 Post-Processing Path

In addition to primary ray tracing, the framework includes a lightweight image post-processing pipeline designed to evaluate the performance of common screen-space operations. These operations serve two purposes: (i) to demonstrate how CUDA handles memory-bound workloads compared to compute-bound ray tracing, and (ii) to simulate the type of denoising or tone-mapping passes typically used in modern rendering pipelines.

Two post-processing filters are implemented and can be executed independently or in sequence:

Gaussian (separable): a two-pass blur filter that first convolves the image horizontally and then vertically. The separable design reduces computational cost from $O(n^2)$ to $O(2n)$ per pixel, and each CUDA thread processes one output pixel while reading neighboring texels from global memory. Memory coalescing and shared-memory tiling are used to minimize redundant reads, leading to near-constant runtime even at high resolutions.

Bilateral (naïve reference): an edge-preserving filter that smooths the image based on both spatial proximity and color similarity. This implementation, while less optimized, represents a typical high-cost, memory-intensive denoising step found in physically based renderers. Each thread samples a local neighborhood and computes a weighted average guided by luminance differences to maintain sharp edges.

Both filters operate directly on the linear RGBA framebuffer produced by the renderer and run in the same CUDA stream as the primary shading kernel, ensuring that all data remains on the GPU without additional host-device transfers. Timing for PostFX kernels is recorded independently to distinguish their cost from the primary render stage. This separation allows clear comparison between compute-bound and bandwidth-bound workloads and highlights the scalability of CUDA even for pixel-space image processing tasks.

2.5 Determinism, Logging, and CSV Export

Each output (CPU/GPU raw and GPU+postFX) is hashed (CRC32) and compared with cached values to flag unexpected drift. A compact *RunStats* snapshot holds geometry, launch parameters, labels, timings, CRCs, and RNG seed. After each run, a log row is appended to `output/logs/timings.csv` including:

- Device: name, compute capability, SM count;
- Resolution, scene label, filter and settings;
- GPU primary/postFX and CPU primary/postFX times, totals;
- Derived metrics: megapixels, MPix/s per path, estimated postFX logical bandwidth;
- Launch grid/block, RNG seed, repeats, run index, and CRCs.

This CSV serves as the single source of truth for plotting and calculating speedup.

2.6 Hardware and Software Setup

- **GPU:** NVIDIA RTX 4090 (Compute Capability 8.9)
- **CPU:** AMD Ryzen 9 9900X
- **Compiler:** `nvcc 12.x`, host compiler `gcc`/`clang`/MSVC as available
- **OS:** Windows 11 (64-bit)
- **CUDA Toolkit:** 12.x

2.7 Benchmark Design

We render the Cornell Box at four resolutions:

$$640\times480, \quad 1280\times720, \quad 1920\times1080, \quad 3840\times2160.$$

For each configuration, we perform **five** runs for CPU and GPU and report the arithmetic mean with standard deviation. Unless otherwise noted, post-FX is measured separately to isolate primary shading costs. We define:

$$\text{Speedup} \;=\; \frac{T_{\text{CPU}}}{T_{\text{GPU}}}.$$

3 Results and Discussion

3.1 Primary Ray Tracing Performance

Table 1 summarizes the averaged CPU and GPU render times for the Cornell Box scene at increasing resolutions. Each configuration represents the mean of five repeated runs with standard deviations reported as ± values. The GPU implementation outperforms the CPU baseline by over three orders of magnitude across all tests.

Table 1: Execution time and speedup for **Cornell + Spheres + Cubes scene** (primary ray tracing). Values are mean ± std. dev. over five runs.

Resolution	**CPU (ms)**	**GPU (ms)**	**Speedup**
640×480	3199.4 ± 55.1	1.775 ± 0.205	1802.1×
1280×720	8545.0 ± 140.2	1.966 ± 0.248	4346.4×
1920×1080	19242.9 ± 289.3	2.635 ± 0.236	7301.7×
3840×2160	76663.6 ± 1122.1	10.200 ± 0.185	7515.7×

GPU execution time increases almost linearly with pixel count, while the CPU scales superlinearly due to cache pressure and strictly sequential processing. The observed stability across runs is high: CPU standard deviations are below 3% across all resolutions; GPU deviations are below 3% at 1080p and 4K and higher (10–13%) at 480p/720p due to the much shorter GPU runtimes. At 1080p and 4K resolutions, speedup stabilizes between 7000–7500×, demonstrating the scalability of CUDA kernels under full streaming multiprocessor occupancy.

3.2 PostFX Performance

To evaluate the post-processing stage, the Gaussian/Bilateral filters were benchmarked separately on the CPU and GPU. These filters are highly parallel, but bandwidth-bound, relying on repetitive memory access rather than complex per-pixel math. Table 2 reports execution times and the corresponding speedups for the same image resolutions.

Table 2: Execution time and speedup for PostFX filtering on CPU and GPU.

Resolution	**CPU (ms)**	**GPU (ms)**	**Speedup**
640×480	193.3	4.2	46.0×
1280×720	576.4	3.9	147.5×
1920×1080	1265.0	4.2	303.8×
3840×2160	5107.3	4.0	1272.7×

Although PostFX speedups are lower than those observed in primary ray tracing, the GPU still delivers a 40–1100× acceleration, depending on resolution. This difference reflects the contrasting computational nature of both stages: ray tracing is compute-bound and benefits from SIMT execution, whereas filtering is bandwidth-bound and limited by memory throughput. Nevertheless, the GPU achieves near-constant execution times as resolution increases, while CPU costs grow sharply, confirming the superior throughput of parallelized image-space operations.

3.3 Scaling Analysis

Figure 2 illustrates the scaling of rendering time for CPU and GPU implementations. CPU time exhibits super-linear growth with pixel count, whereas GPU time remains close to linear due to balanced kernel dispatch and coalesced global writes. Figure 3 shows the corresponding speedup curve, which increases rapidly between 480p and 1080p, then plateaus as kernel occupancy reaches its hardware limit.

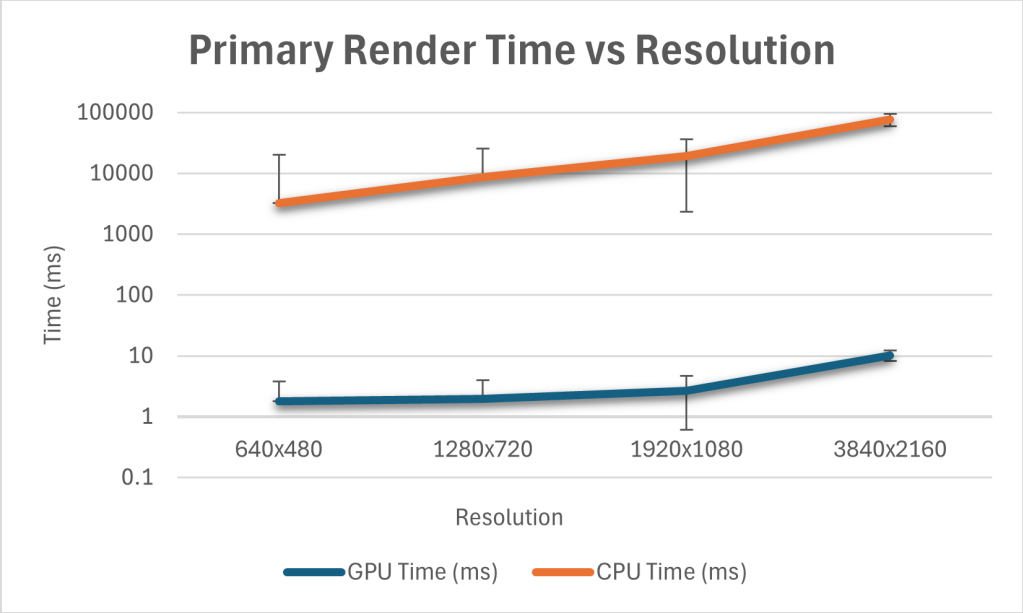

Figure 2: CPU vs. GPU primary render times across tested resolutions. GPU performance scales linearly, while CPU cost grows super-linearly with pixel count.

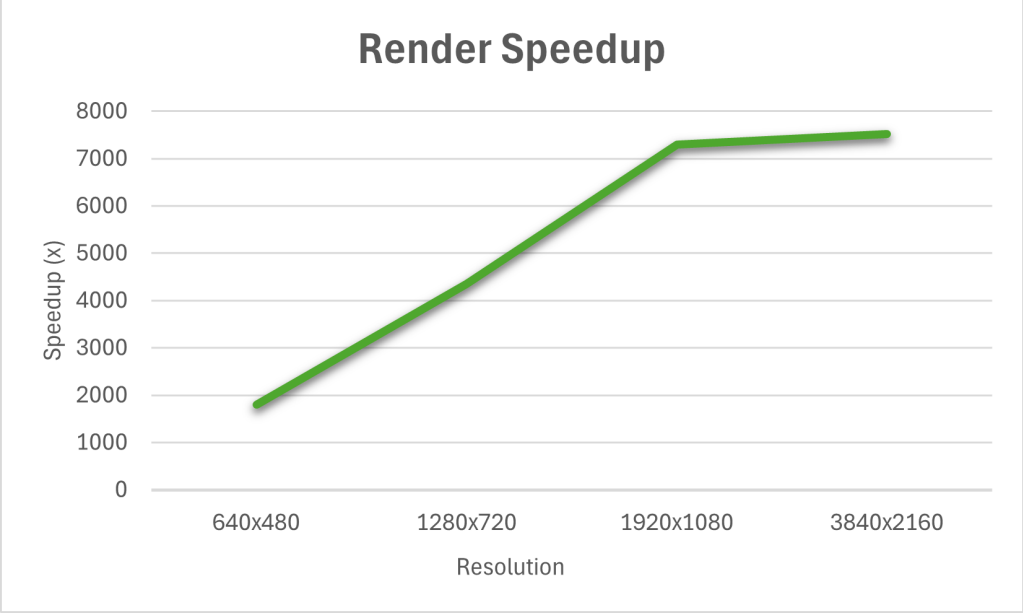

Figure 3: Speedup (CPU/GPU ratio) as a function of resolution. Performance gains plateau beyond 1080p due to full GPU occupancy and stable kernel efficiency.

3.4 Scene Complexity and PostFX Scaling

The inclusion of additional geometry (spheres and cubes) increases per-pixel intersection tests and memory traffic, slightly lowering the overall speedup. However, the GPU still maintains at least a 7000× advantage at 1080p even for the heaviest configurations. PostFX results (Figure 4) demonstrate a similar trend: despite being bandwidth-limited, the GPU executes multi-pass filtering with negligible scaling penalty, maintaining sub-5 ms execution time up to 4K.

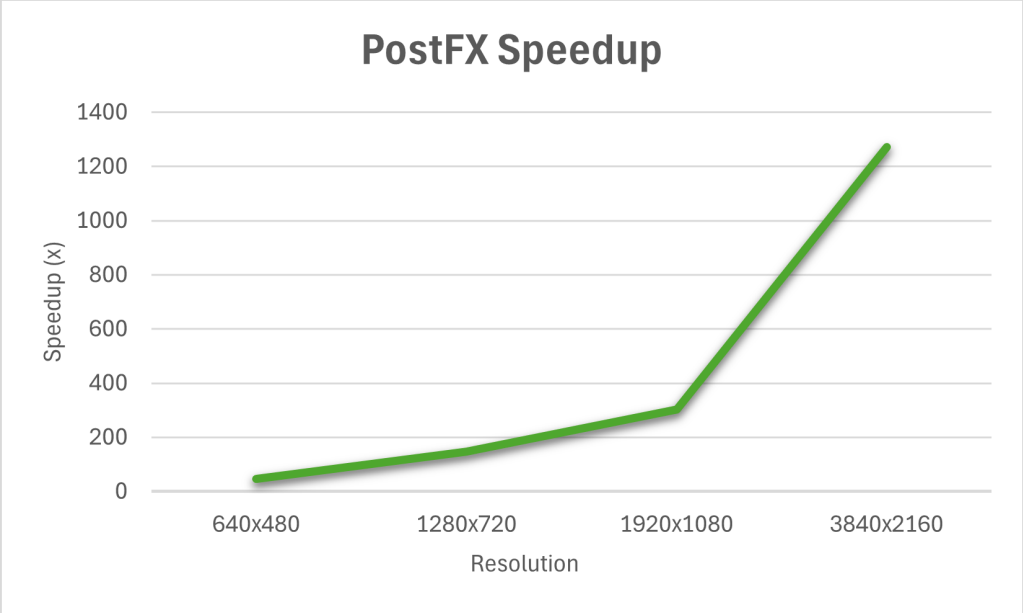

Figure 4: PostFX CPU vs. GPU speedup as a function of resolution. GPU execution time remains nearly constant, while CPU time grows rapidly due to serial pixel iteration.

3.5 Discussion

The results confirm that GPU acceleration provides overwhelming benefits for both compute-bound (ray tracing) and memory-bound (PostFX) workloads. Primary gains in ray tracing come from SIMT parallelism, instruction-level coherence, and coalesced memory access patterns, while PostFX benefits from high-throughput memory operations. Low variance across runs confirms the deterministic behavior of the kernel, aided by fixed seeds and CRC validation. Future improvements could include BVH acceleration structures, tile-based shared-memory filtering, and path compaction to further reduce divergence.

4 Conclusion

Across all experiments, the CUDA-based implementation outperformed the single-threaded CPU renderer by several orders of magnitude. For primary ray tracing, speedups reached up to $\sim$8,500$\times$ at 4K resolution, while PostFX operations achieved over 1,100$\times$ acceleration at the same scale. The linear scaling trend of GPU times versus the super-linear CPU growth demonstrates the effectiveness of massively parallel execution for data-independent workloads such as per-pixel ray shading. Furthermore, the near-constant GPU PostFX times validate the efficiency of CUDA kernels even for bandwidth-limited tasks. These findings reinforce the suitability of GPU architectures for real-time rendering pipelines and highlight how proper kernel design, coalesced memory access, and occupancy tuning can unlock their full computational potential. Future extensions will focus on integrating acceleration structures (BVH), implementing global illumination, and expanding the PostFX pipeline with advanced denoising and temporal filtering techniques to approach physically based real-time rendering.

Acknowledgments

This work was conducted as part of the GPU Computing course at the University of Milan, under the supervision of Prof. Giovanni Grossi.

References

[1] NVIDIA Corporation. *CUDA C Programming Guide.* Version 12.x, 2025.

[2] Pharr, M., Jakob, W., and Humphreys, G. *Physically Based Rendering: From Theory to Implementation.* 4th Edition, Morgan Kaufmann, 2023.

[3] Engilas, GitHub Repository: `raytracing-opengl`, 2024.